

Course Machine Learning

Part II – Supervised Learning

Course #9 – Clustering

**Professor
Abdelouahab Moussaoui**



1 — Unsupervised Machine Learning

In the previous topic, we learned supervised machine learning in which models are trained using labeled data under the supervision of training data. But there may be many cases in which we do not have labeled data and need to find the hidden patterns from the given dataset. So, to solve such types of cases in machine learning, we need unsupervised learning techniques.

What is Unsupervised Learning?

As the name suggests, unsupervised learning is a machine learning technique in which models are not supervised using training dataset. Instead, models itself find the hidden patterns and insights from the given data. It can be compared to learning which takes place in the human brain while learning new things. It can be defined as:

Unsupervised learning is a type of machine learning in which models are trained using unlabeled dataset and are allowed to act on that data without any supervision.

Unsupervised learning cannot be directly applied to a regression or classification problem because unlike supervised learning, we have the input data but no corresponding output data. The goal of unsupervised learning is to **find the underlying structure of dataset, group that data according to similarities, and represent that dataset in a compressed format.**

Example: Suppose the unsupervised learning algorithm is given an input dataset containing images of different types of cats and dogs. The algorithm is never trained upon the given dataset, which means it does not have any idea about the features of the dataset. The task

of the unsupervised learning algorithm is to identify the image features on their own. Unsupervised learning algorithm will perform this task by clustering the image dataset into the groups according to similarities between images.



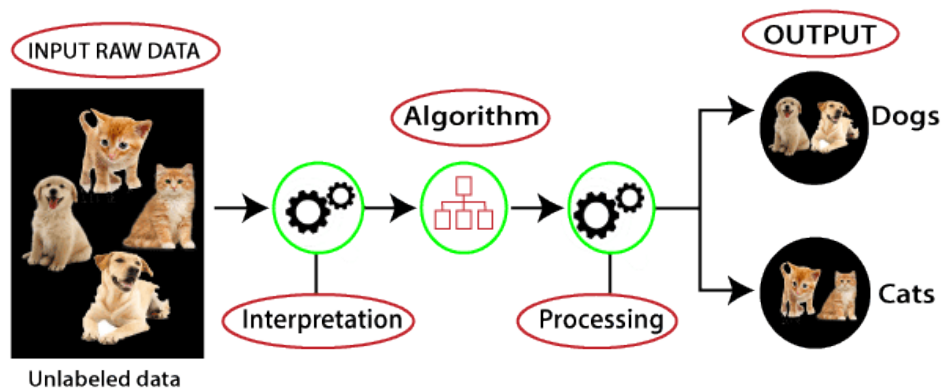
Why use Unsupervised Learning?

Below are some main reasons which describe the importance of Unsupervised Learning:

- Unsupervised learning is helpful for finding useful insights from the data.
- Unsupervised learning is much similar as a human learns to think by their own experiences, which makes it closer to the real AI.
- Unsupervised learning works on unlabeled and uncategorized data which make unsupervised learning more important.
- In real-world, we do not always have input data with the corresponding output so to solve such cases, we need unsupervised learning.

Working of Unsupervised Learning

Working of unsupervised learning can be understood by the below diagram:

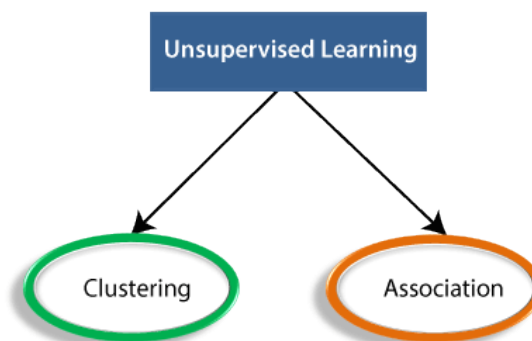


Here, we have taken an unlabeled input data, which means it is not categorized and corresponding outputs are also not given. Now, this unlabeled input data is fed to the machine learning model in order to train it. Firstly, it will interpret the raw data to find the hidden patterns from the data and then will apply suitable algorithms such as k-means clustering, Decision tree, etc.

Once it applies the suitable algorithm, the algorithm divides the data objects into groups according to the similarities and difference between the objects.

Types of Unsupervised Learning Algorithm:

The unsupervised learning algorithm can be further categorized into two types of problems:



- **Clustering:** Clustering is a method of grouping the objects into clusters such that objects with most similarities remains into a group and has less or no similarities with the objects of another group. Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.
- **Association:** An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. It determines the set of items that occurs together in the dataset. Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis.

Note: We will learn these algorithms in later chapters.

Unsupervised Learning algorithms:

Below is the list of some popular unsupervised learning algorithms:

- K-means clustering
- KNN (k-nearest neighbors)
- Hierarchical clustering
- Anomaly detection
- Neural Networks
- Principle Component Analysis
- Independent Component Analysis
- Apriori algorithm
- Singular value decomposition

Advantages of Unsupervised Learning

- Unsupervised learning is used for more complex tasks as compared to supervised learning because, in unsupervised learning, we don't have labeled input data.
- Unsupervised learning is preferable as it is easy to get unlabeled data in comparison to labeled **data**.

Disadvantages of Unsupervised Learning

- Unsupervised learning is intrinsically more difficult than supervised learning as it does not have corresponding output.
 - The result of the unsupervised learning algorithm might be less accurate as input data is not labeled, and algorithms do not know the exact output in advance.
-

2 — Clustering

Introduction to Clustering

It is basically a type of unsupervised learning method. An unsupervised learning method is a method in which we draw references from datasets consisting of input data without labeled responses. Generally, it is used as a process to find meaningful structure, explanatory underlying processes, generative features, and groupings inherent in a set of examples.

Clustering is the task of dividing the population or data points into a number of groups such that data points in the same groups are more similar to other data points in the same group and dissimilar to the data points in other groups. It is basically a collection of objects on the basis of similarity and dissimilarity between them.

Clustering or cluster analysis is a machine learning technique, which groups the unlabelled dataset. It can be defined as **"A way of grouping the data points into different clusters, consisting of similar data points. The objects with the possible similarities remain in a group that has less or no similarities with another group."**

It does it by finding some similar patterns in the unlabelled dataset such as shape, size, color, behavior, etc., and divides them as per the presence and absence of those similar patterns.

It is an [unsupervised learning](#) method, hence no supervision is provided to the algorithm, and it deals with the unlabeled dataset.

After applying this clustering technique, each cluster or group is provided with a cluster-ID. ML system can use this id to simplify the processing of large and complex datasets

The clustering technique is commonly used for **statistical data analysis**.



Note: Clustering is somewhere similar to the [classification algorithm](#), but the difference is the type of dataset that we are using. In classification, we work with the labeled data set, whereas in clustering, we work with the unlabelled dataset.

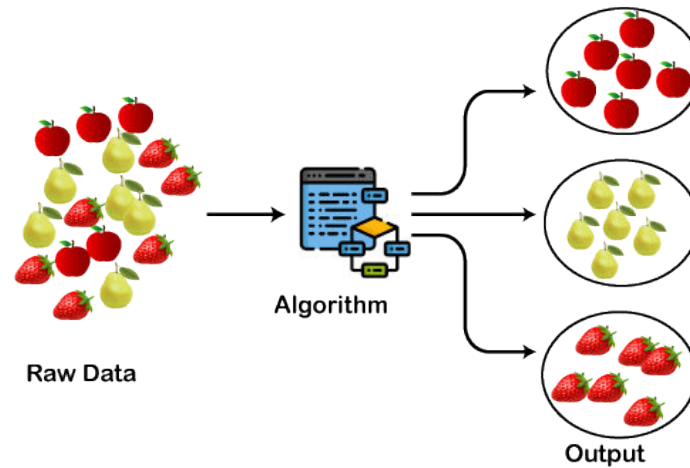
Example: Let's understand the clustering technique with the real-world example of Mall: When we visit any shopping mall, we can observe that the things with similar usage are grouped together. Such as the t-shirts are grouped in one section, and trousers are at other sections, similarly, at vegetable sections, apples, bananas, Mangoes, etc., are grouped in separate sections, so that we can easily find out the things. The clustering technique also works in the same way. Other examples of clustering are grouping documents according to the topic.

The clustering technique can be widely used in various tasks. Some most common uses of this technique are:

- Market Segmentation
- Statistical data analysis
- Social network analysis
- Image segmentation
- Anomaly detection, etc.

Apart from these general usages, it is used by the **Amazon** in its recommendation system to provide the recommendations as per the past search of products. **Netflix** also uses this technique to recommend the movies and web-series to its users as per the watch history.

The below diagram explains the working of the clustering algorithm. We can see the different fruits are divided into several groups with similar properties.



Types of Clustering Methods

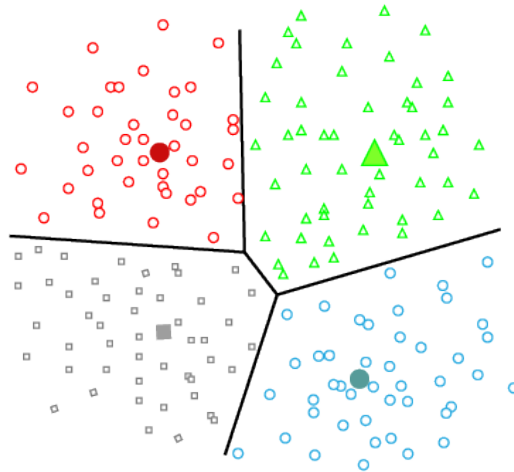
The clustering methods are broadly divided into **Hard clustering** (datapoint belongs to only one group) and **Soft Clustering** (data points can belong to another group also). But there are also other various approaches of Clustering exist. Below are the main clustering methods used in Machine learning:

1. Partitioning Clustering
2. Density-Based Clustering
3. Distribution Model-Based Clustering
4. Hierarchical Clustering
5. Fuzzy Clustering

Partitioning Clustering

It is a type of clustering that divides the data into non-hierarchical groups. It is also known as the **centroid-based method**. The most common example of partitioning clustering is the [*K-Means Clustering algorithm*](#).

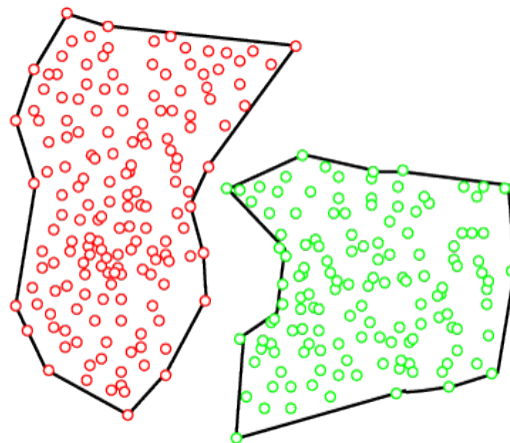
In this type, the dataset is divided into a set of k groups, where K is used to define the number of pre-defined groups. The cluster center is created in such a way that the distance between the data points of one cluster is minimum as compared to another cluster centroid.



Density-Based Clustering

The density-based clustering method connects the highly-dense areas into clusters, and the arbitrarily shaped distributions are formed as long as the dense region can be connected. This algorithm does it by identifying different clusters in the dataset and connects the areas of high densities into clusters. The dense areas in data space are divided from each other by sparser areas.

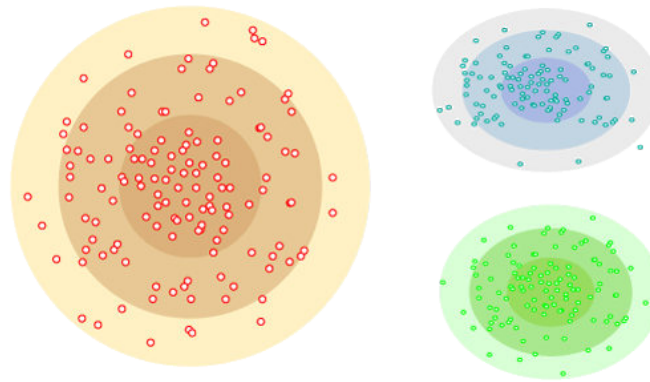
These algorithms can face difficulty in clustering the data points if the dataset has varying densities and high dimensions.



Distribution Model-Based Clustering

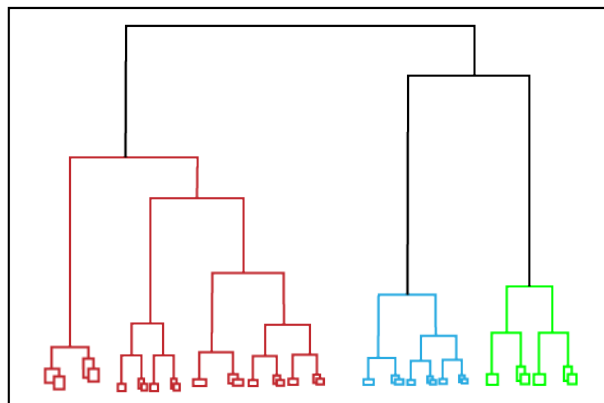
In the distribution model-based clustering method, the data is divided based on the probability of how a dataset belongs to a particular distribution. The grouping is done by assuming some distributions commonly **Gaussian Distribution**.

The example of this type is the **Expectation-Maximization Clustering algorithm** that uses Gaussian Mixture Models (GMM).



Hierarchical Clustering

Hierarchical clustering can be used as an alternative for the partitioned clustering as there is no requirement of pre-specifying the number of clusters to be created. In this technique, the dataset is divided into clusters to create a tree-like structure, which is also called a **dendrogram**. The observations or any number of clusters can be selected by cutting the tree at the correct level. The most common example of this method is the **Agglomerative Hierarchical algorithm**.



Fuzzy Clustering

Fuzzy clustering is a type of soft method in which a data object may belong to more than one group or cluster. Each dataset has a set of membership coefficients, which depend on the degree of membership to be in a cluster. **Fuzzy C-means algorithm** is the example of this type of clustering; it is sometimes also known as the Fuzzy k-means algorithm.

Types of Clustering Algorithms

The Clustering algorithms can be divided based on their models that are explained above. There are different types of clustering algorithms published, but only a few are commonly used. The clustering algorithm is based on the kind of data that we are using. Such as, some algorithms need to guess the number of clusters in the given dataset, whereas some are required to find the minimum distance between the observation of the dataset.

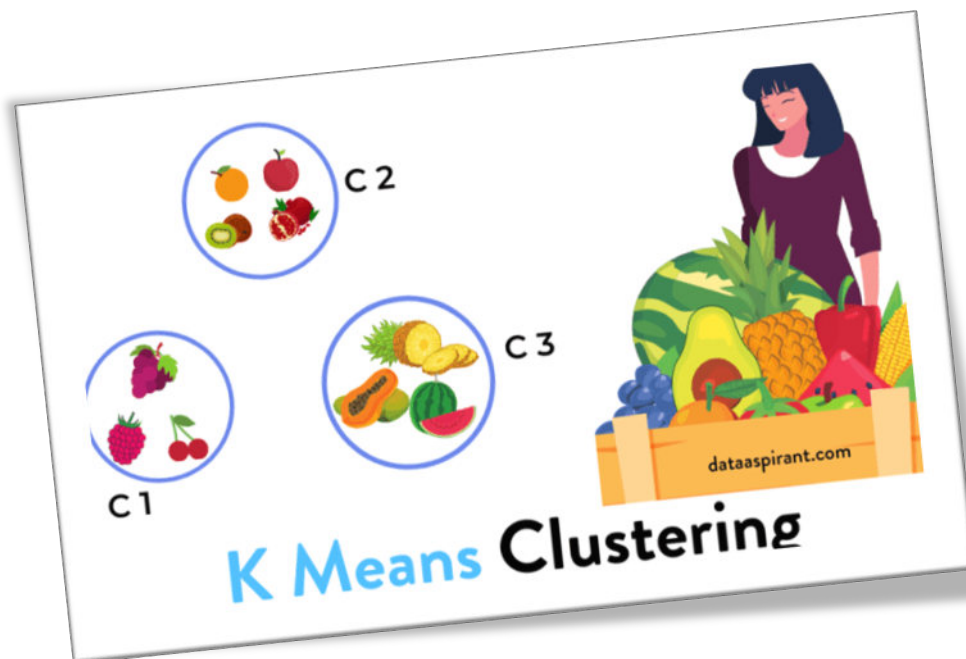
Here we are discussing mainly popular Clustering algorithms that are widely used in machine learning:

1. **K-Means algorithm:** The k-means algorithm is one of the most popular clustering algorithms. It classifies the dataset by dividing the samples into different clusters of equal variances. The number of clusters must be specified in this algorithm. It is fast with fewer computations required, with the linear complexity of $O(n)$.
2. **Mean-shift algorithm:** Mean-shift algorithm tries to find the dense areas in the smooth density of data points. It is an example of a centroid-based model, that works on updating the candidates for centroid to be the center of the points within a given region.
3. **DBSCAN Algorithm:** It stands for **Density-Based Spatial Clustering of Applications with Noise**. It is an example of a density-based model similar to the mean-shift, but with some remarkable advantages. In this algorithm, the areas of high density are separated by the areas of low density. Because of this, the clusters can be found in any arbitrary shape.
4. **Expectation-Maximization Clustering using GMM:** This algorithm can be used as an alternative for the k-means algorithm or for those cases where K-means can be failed. In GMM, it is assumed that the data points are Gaussian distributed.
5. **Agglomerative Hierarchical algorithm:** The Agglomerative hierarchical algorithm performs the bottom-up hierarchical clustering. In this, each data point is treated as a single cluster at the outset and then successively merged. The cluster hierarchy can be represented as a tree-structure.
6. **Affinity Propagation:** It is different from other clustering algorithms as it does not require to specify the number of clusters. In this, each data point sends a message between the pair of data points until convergence. It has $O(N^2T)$ time complexity, which is the main drawback of this algorithm.

Applications of Clustering

Below are some commonly known applications of clustering technique in Machine Learning:

- **In Identification of Cancer Cells:** The clustering algorithms are widely used for the identification of cancerous cells. It divides the cancerous and non-cancerous data sets into different groups.
- **In Search Engines:** Search engines also work on the clustering technique. The search result appears based on the closest object to the search query. It does it by grouping similar data objects in one group that is far from the other dissimilar objects. The accurate result of a query depends on the quality of the clustering algorithm used.
- **Customer Segmentation:** It is used in market research to segment the customers based on their choice and preferences.
- **In Biology:** It is used in the biology stream to classify different species of plants and animals using the image recognition technique.
- **In Land Use:** The clustering technique is used in identifying the area of similar lands use in the GIS database. This can be very useful to find that for what purpose the particular land should be used, that means for which purpose it is more suitable.



2 — K-Means Clustering Algorithm

How does K-Means work?

The K-means algorithm is an iterative process with three critical stages:

1. Pick initial cluster centroids

The algorithm starts by picking initial **k** cluster centers which are known as centroids. Determining the optimal number of clusters i.e **k** as well as proper selection of the initial clusters is extremely important for the performance of the model. The number of clusters should always be dependent on the nature of the dataset while poor selection of the initial cluster can lead to the problem of local convergence. Thankfully, we have solutions for both.

For selection of initial clusters, we can either run multiple iterations of the model with various initializations to pick the most stable one or use the "**k-means++**" algorithm which has the following steps:

- *Randomly select the first centroid from the dataset*
- *Compute distance of all points in the dataset from the selected centroid*
- *Pick a point as the new centroid that has maximum probability proportional to this distance*
- *Repeat steps 2 and 3 until k centroids have been sampled*

The algorithm initializes the centroids to be distant from each other leading to more stable results than random initialization.

2. Cluster assignment

K-means assigns the data points to the closest cluster centroids based on Euclidean distance between the point and all centroids.

3. Move centroid

The model finally calculates the average of all the points in a cluster and moves the centroid to that average location.

Step 2 and 3 are repeated until there is **no change** in the clusters or possibly some other stopping condition is met (like maximum number of iterations).

For implementing the model in python or R, we need to do specify the number of clusters first. We can use the [elbow method](#), [Gap Statistic](#), [Silhouette score](#), [Calinski Harabasz score](#) and [Davies Bouldin score](#). For each of these methods the optimal number of clusters are as follows:

- [Elbow method](#)
- [Gap statistic](#)
- [Silhouette score](#)
- [Calinski Harabasz score](#)
- [Davies Bouldin score](#)

K-Means Clustering Algorithm

K-Means Clustering is an unsupervised learning algorithm that is used to solve the clustering problems in machine learning or data science. In this topic, we will learn what is K-means clustering algorithm, how the algorithm works, along with the Python implementation of k-means clustering.

What is K-Means Algorithm?

[K-Means](#) Clustering is an [Unsupervised Learning algorithm](#), which groups the unlabeled dataset into different clusters. Here K defines the number of pre-defined clusters that need to be created in the process, as if $K=2$, there will be two clusters, and for $K=3$, there will be three clusters, and so on.

It is an iterative algorithm that divides the unlabeled dataset into k different clusters in such a way that each dataset belongs only one group that has similar properties.

It allows us to cluster the data into different groups and a convenient way to discover the categories of groups in the unlabeled dataset on its own without the need for any training.

It is a centroid-based algorithm, where each cluster is associated with a centroid. The main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters.

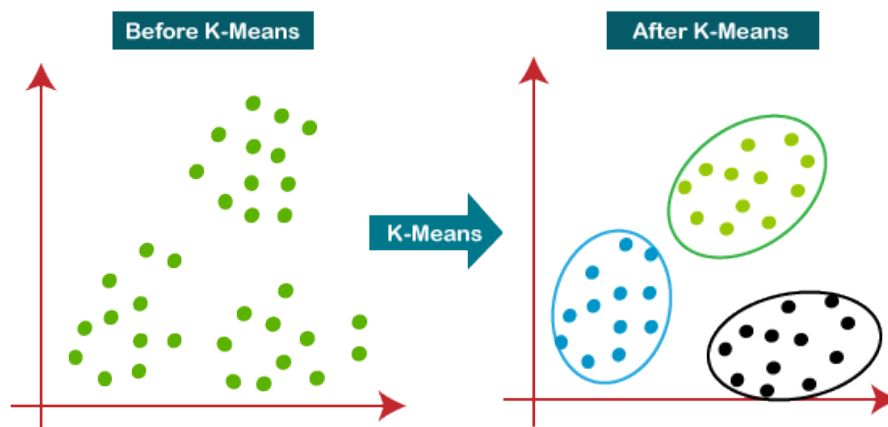
The algorithm takes the unlabeled dataset as input, divides the dataset into k-number of clusters, and repeats the process until it does not find the best clusters. The value of k should be predetermined in this algorithm.

The k-means [clustering](#) algorithm mainly performs two tasks:

- Determines the best value for K center points or centroids by an iterative process.
- Assigns each data point to its closest k-center. Those data points which are near to the particular k-center, create a cluster.

Hence each cluster has datapoints with some commonalities, and it is away from other clusters.

The below diagram explains the working of the K-means Clustering Algorithm:



How does the K-Means Algorithm Work?

The working of the K-Means algorithm is explained in the below steps:

Step-1: Select the number K to decide the number of clusters.

Step-2: Select random K points or centroids. (It can be other from the input dataset).

Step-3: Assign each data point to their closest centroid, which will form the predefined K clusters.

Step-4: Calculate the variance and place a new centroid of each cluster.

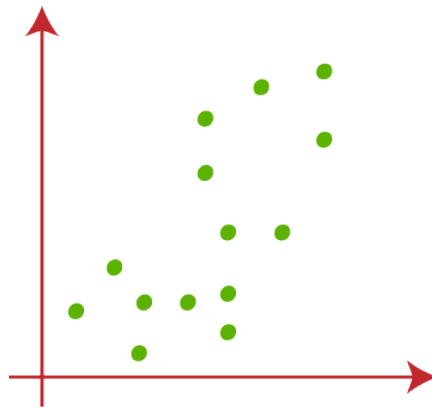
Step-5: Repeat the third steps, which means reassign each datapoint to the new closest centroid of each cluster.

Step-6: If any reassignment occurs, then go to step-4 else go to FINISH.

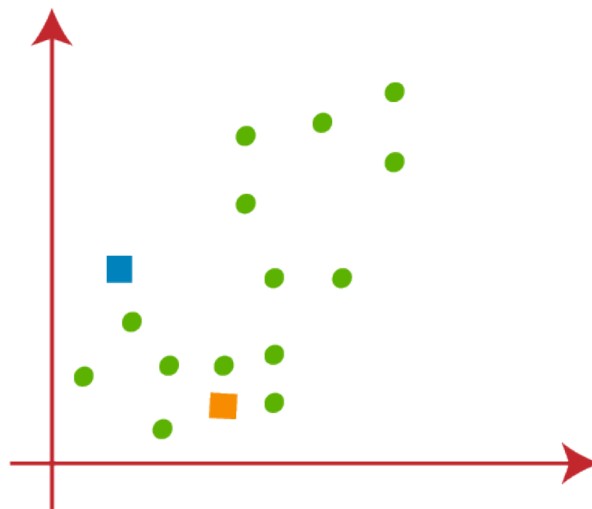
Step-7: The model is ready.

Let's understand the above steps by considering the visual plots:

Suppose we have two variables M1 and M2. The x-y axis scatter plot of these two variables is given below:

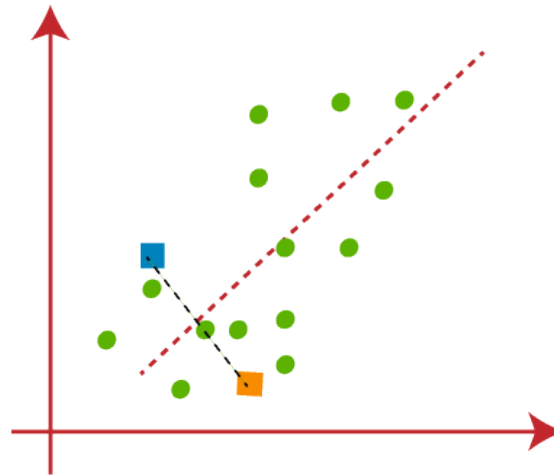


- Let's take number k of clusters, i.e., $K=2$, to identify the dataset and to put them into different clusters. It means here we will try to group these datasets into two different clusters.
- We need to choose some random k points or centroid to form the cluster. These points can be either the points from the dataset or any other point. So, here we are selecting the below two points as k points, which are not the part of our dataset. Consider the below image:

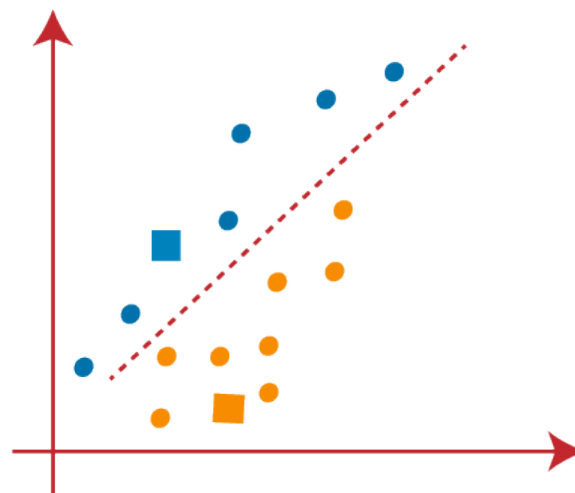


- Now we will assign each data point of the scatter plot to its closest K -point or centroid. We will compute it by applying some mathematics that we have studied to calculate the distance between two points. So, we will draw a median between both the centroids. Consider the below

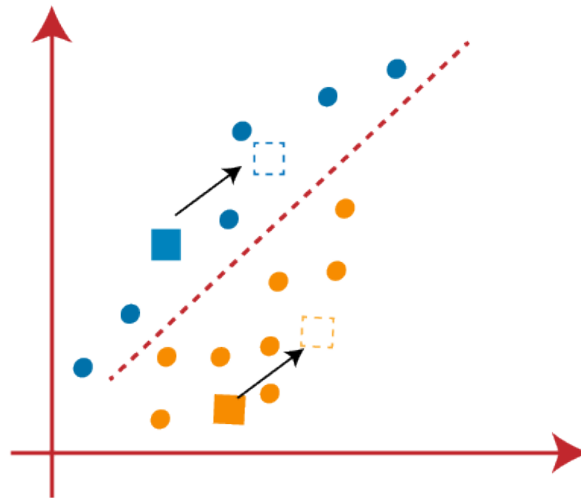
image:



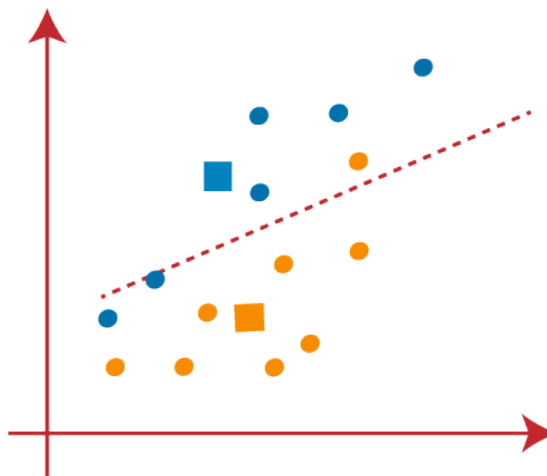
From the above image, it is clear that points left side of the line is near to the K1 or blue centroid, and points to the right of the line are close to the yellow centroid. Let's color them as blue and yellow for clear visualization.



- As we need to find the closest cluster, so we will repeat the process by choosing a new centroid. To choose the new centroids, we will compute the center of gravity of these centroids, and will find new centroids as below:

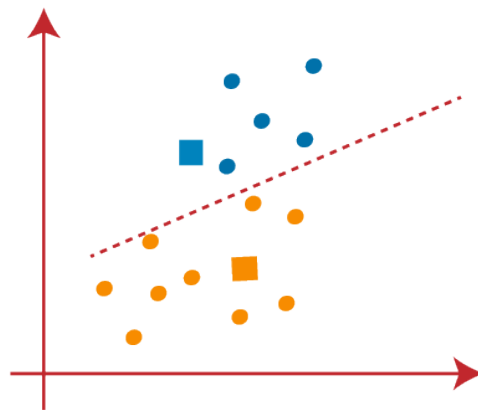


- Next, we will reassign each datapoint to the new centroid. For this, we will repeat the same process of finding a median line. The median will be like below image:



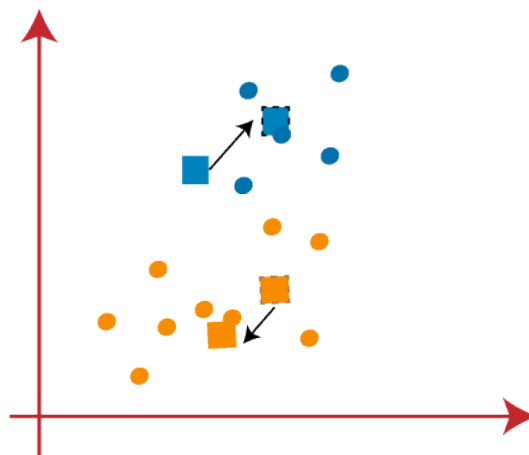
○

From the above image, we can see, one yellow point is on the left side of the line, and two blue points are right to the line. So, these three points will be assigned to new centroids.

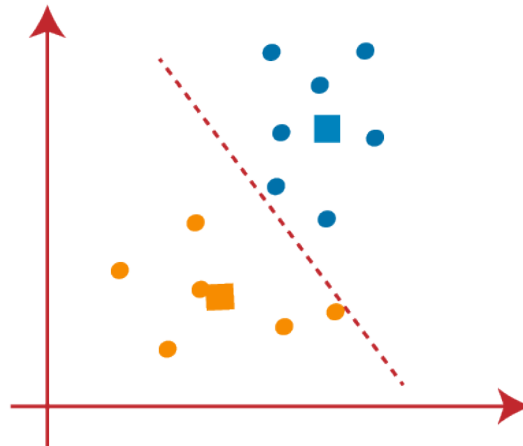


As reassignment has taken place, so we will again go to the step-4, which is finding new centroids or K-points.

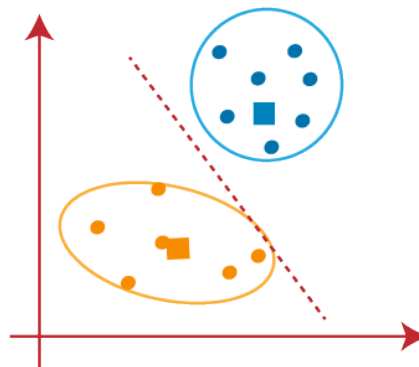
- We will repeat the process by finding the center of gravity of centroids, so the new centroids will be as shown in the below image:



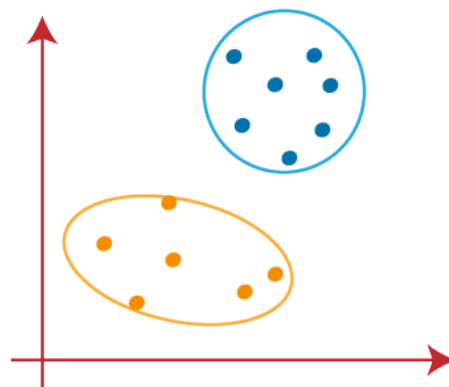
- As we got the new centroids so again will draw the median line and reassign the data points. So, the image will be:



- We can see in the above image; there are no dissimilar data points on either side of the line, which means our model is formed. Consider the below image:



As our model is ready, so we can now remove the assumed centroids, and the two final clusters will be as shown in the below image:



Hyperparameters



Hyperparameters are model configurations properties that define the model and remain constants during the training of the model. The design of the model can be changed by tuning the hyperparameters. For K-Means clustering there are 3 main hyperparameters to set-up to define the best configuration of the model:

- *Initial values of clusters*
- *Distance measures*
- *Number of clusters*

Initial values of clusters greatly impact the clustering model, there are various algorithms to initialize the values. Distance measures are used to find points in clusters to the cluster center, different distance measures yield different clusters.

How to choose the value of "K number of clusters" in K-means Clustering?

The performance of the K-means clustering algorithm depends upon highly efficient clusters that it forms. But choosing the optimal number of clusters is a big task. There are some different ways to find the optimal number of clusters, but here we are discussing the most appropriate method to find the number of clusters or value of K.

The number of clusters (**k**) is the most important hyperparameter in K-Means clustering. If we already know beforehand, the number of clusters to group the data into, then there is no use to tune the value of k. For example, k=10 for the MNIST digit classification dataset.

If there is no idea about the optimal value of k, then there are various methods to find the optimal/best value of k. In this course we will cover two such methods:

- **Elbow Method**
- **Silhouette Method**

Elbow Method

The Elbow method is one of the most popular ways to find the optimal number of clusters. This method uses the concept of WCSS value. **WCSS** stands for **Within Cluster Sum of Squares**, which defines the total variations within a cluster. The formula to calculate the value of WCSS (for 3 clusters) is given below:

$$WCSS = \sum_{P_i \text{ in Cluster1}} \text{distance}(P_i, C_1)^2 + \sum_{P_i \text{ in Cluster2}} \text{distance}(P_i, C_2)^2 + \sum_{P_i \text{ in Cluster3}} \text{distance}(P_i, C_3)^2$$

In the above formula of WCSS,

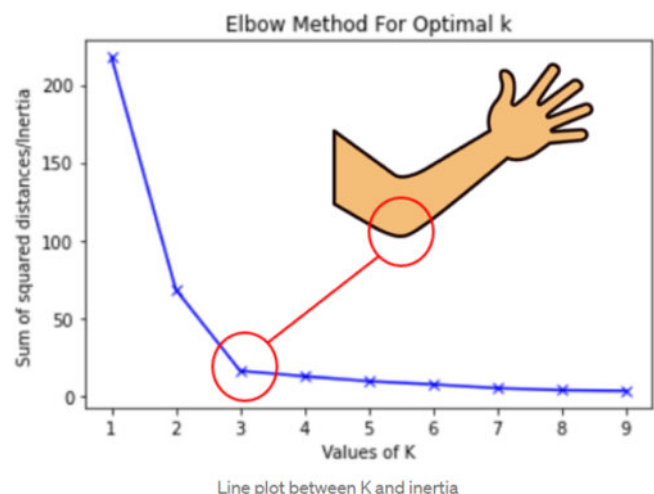
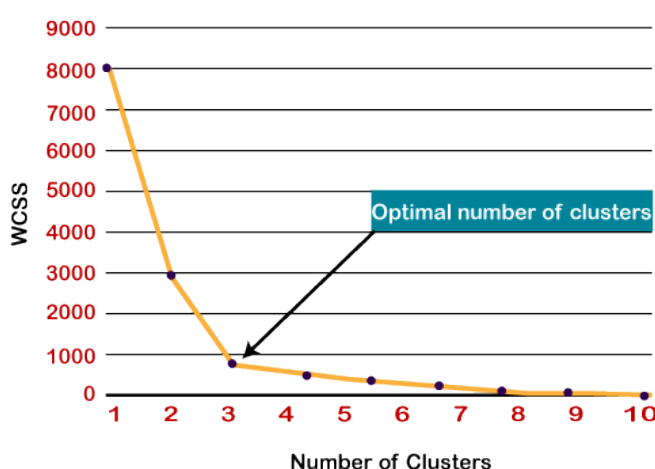
$\sum_{P_i \text{ in Cluster1}} \text{distance}(P_i, C_1)^2$: It is the sum of the square of the distances between each data point and its centroid within a cluster1 and the same for the other two terms.

To measure the distance between data points and centroid, we can use any method such as Euclidean distance or Manhattan distance.

To find the optimal value of clusters, the elbow method follows the below steps:

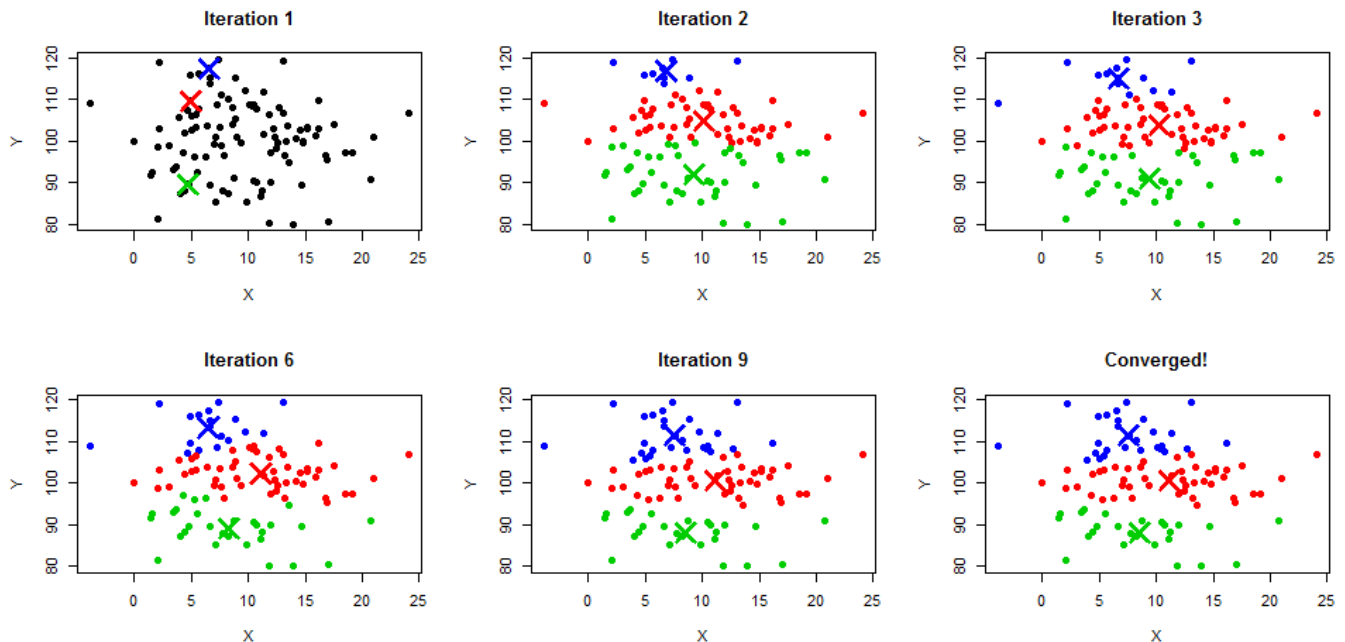
- It executes the K-means clustering on a given dataset for different K values (ranges from 1-10).
- For each value of K, calculates the WCSS value.
- Plots a curve between calculated WCSS values and the number of clusters K.
- The sharp point of bend or a point of the plot looks like an arm, then that point is considered as the best value of K.

Since the graph shows the sharp bend, which looks like an elbow, hence it is known as the elbow method. The graph for the elbow method looks like the below image:





Note: We can choose the number of clusters equal to the given data points. If we the number of clusters equal to the data points, then the value of WCSS becomes zero, and that will be the endpoint of the plot.

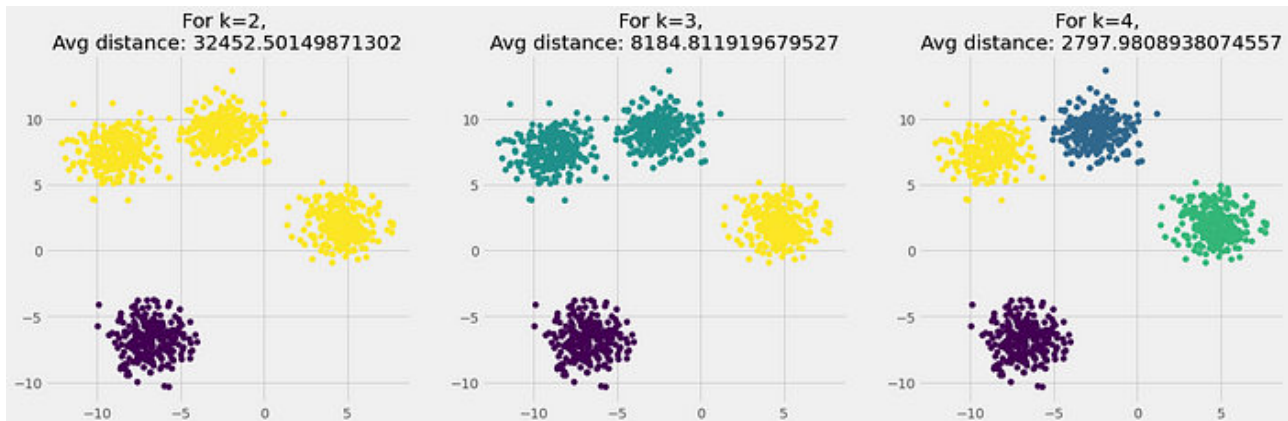


Elbow Method is an empirical method to find the optimal number of clusters for a dataset. In this method, we pick a range of candidate values of k , then apply K-Means clustering using each of the values of k . Find the average distance of each point in a cluster to its centroid, and represent it in a plot. Pick the value of k , where the **average distance falls suddenly**.



With an increase in the number of clusters (k), the average distance decreases. To find the optimal number of clusters (k), observe the plot and find the value of k for which there is a sharp and steep fall of the distance. This will be an optimal point of k where an elbow occurs.

In the above plot, there is a sharp fall of average distance at **k=2, 3, and 4**. Here comes a confusion to pick the best value of k. In the below plot observe the clusters formed for **k=2, 3, and 4** with their average distance.



(Image by Author), Scatter plot of clusters formed at k=2, 3, and 4

This data is 2-D, so it's easy to visualize and pick the best value of k, which is k=4. For higher-dimensional data, we can employ the **Silhouette Method** to find the best k, which is a **better alternative to Elbow Method**.

Elbow Method

The **silhouette Method** is also a method to find the optimal number of clusters and interpretation and validation of consistency within clusters of data. The silhouette method computes silhouette coefficients of each point that measure how much a point is similar to its own cluster compared to other clusters. by providing a **succinct graphical representation** of how well each object has been classified.

Compute **silhouette coefficients** for each of point, and average it out for all the samples to get the **silhouette score**.

The silhouette value is a measure of how similar an object is to its own cluster (**cohesion**) compared to other clusters (**separation**). The value of the silhouette ranges between $[-1, 1]$, where a high value indicates that the object is well matched to its own cluster and poorly matched to neighboring clusters. If most objects have a high value, then the clustering configuration is appropriate. If many points have a low or negative value, then the clustering configuration may have too many or too few clusters.

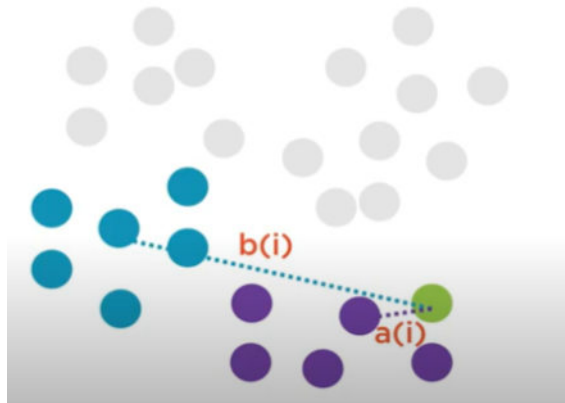
Computing Silhouette Coefficient:

Steps to find the silhouette coefficient of an i 'th point:

1. Compute $a(i)$: The average distance of that point with all other points in the same clusters.

2. Compute $b(i)$: The average distance of that point with all the points in the closest cluster to its cluster.
3. Compute $s(i)$ — silhouette coefficient of i 'th point using below mentioned formula.

$$s(i) = \frac{b(i) - a(i)}{\max(b(i), a(i))}$$



Diagrammatic representation of $a(i)$ and $b(i)$ from the above-mentioned formula to compute silhouette coefficient — $s(i)$. After computing the silhouette coefficient for each point, average it out to get the silhouette score.

Silhouette Analysis:

Silhouette is a measure of how a clustering algorithm has performed. After computing the silhouette coefficient of each point in the dataset, plot it to get a visual representation of how well the dataset is clustered into k clusters. The silhouette plot displays a measure of how close each point in one cluster is to points in the neighboring clusters and thus provides a way to assess parameters like the number of clusters visually. This measure has a range of $[-1, 1]$.



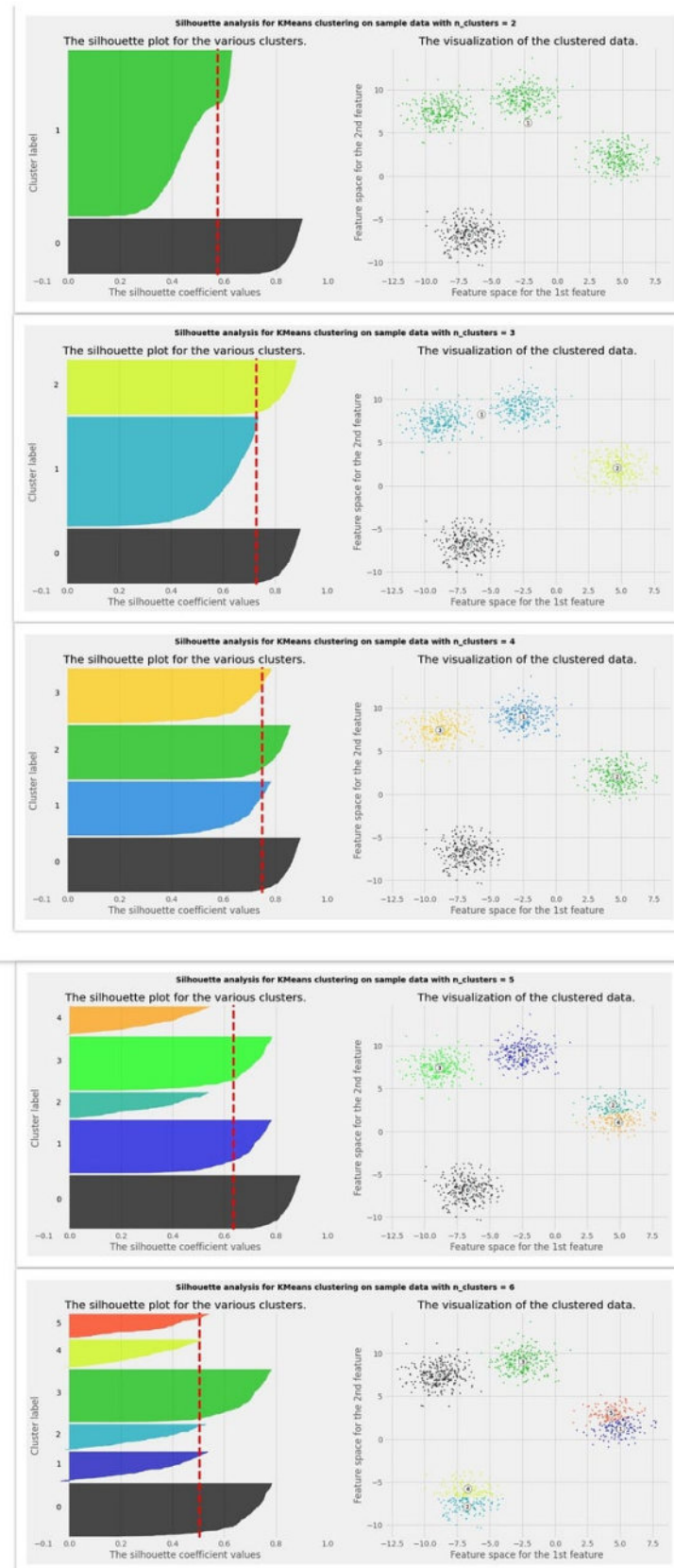
- The Silhouette coefficient of $+1$ indicates that the sample is far away from the neighbouring clusters.
- The Silhouette coefficient of 0 indicates that the sample is on or very close to the decision boundary between two neighbouring clusters.
- Silhouette coefficient <0 indicates that those samples might have been assigned to the wrong cluster or are outliers

Find the optimal value of 'k' using Silhouette Analysis:

Similar to the previous Elbow method, we pick a range of candidate values of k (number of clusters), then train *K-Means* clustering for each of the values of k . For each *k-Means* clustering model represent the silhouette coefficients in a plot and observe the fluctuations and outliers of each cluster.



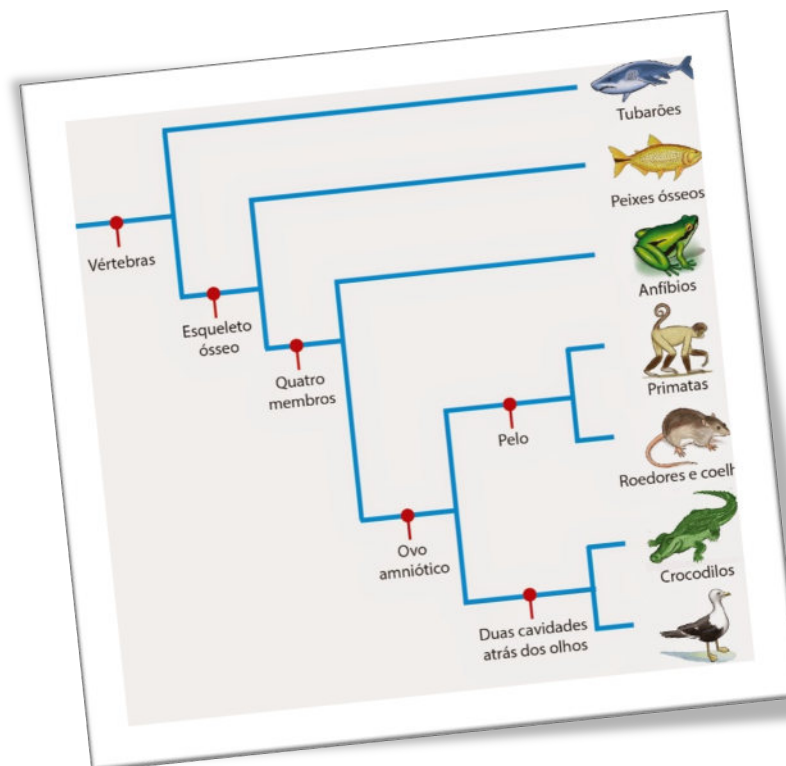
Silhouette score vs the number of clusters



Silhouette Analysis and scatter plot for each cluster in KMeans clustering on entire data with $n_cluster=[2,3,4,5,6]$

Observations from above Silhouette Plots:

- The silhouette plot shows that the *n_cluster* value of **3** is a bad pick, as all the points in the cluster with *cluster_label*=0 are below-average silhouette scores.
- The silhouette plot shows that the *n_cluster* value of **5** is a bad pick, as all the points in the cluster with *cluster_label*=2 and 4 are below-average silhouette scores.
- The silhouette plot shows that the *n_cluster* value of **6** is a bad pick, as all the points in the cluster with *cluster_label*=1,2,4 and 5 are below-average silhouette scores, and also due to the presence of outliers.
- Silhouette analysis is more ambivalent in deciding between 2 and 4.
- The thickness of the silhouette plot for the cluster with *cluster_label*=1 when *n_clusters*=2, is bigger in size owing to the grouping of the 3 sub-clusters into one big cluster.
- For *n_clusters*=4, all the plots are more or less of similar thickness and hence are of similar sizes, as can be considered as **best 'k'**.



3 — Hierarchical Clustering Algorithm

Clustering in Sum

In Data Science, big, messy problem sets are unavoidable. If we keep them as such, every step of the analytical process will be much more cumbersome. Given this, it's inarguable that we would want a way to view our data at large in a logical and organized manner.

Clustering is one of the most popular methods in data science and is an unsupervised Machine Learning technique that enables us to find structures within our data, without trying to obtain specific insight. In cluster analysis, we partition our dataset into groups that share similar attributes. We want to assign our data points to clusters such that there is **"high intra-cluster similarity"** and **"low inter-cluster similarity"**. Here are some examples of real-life applications of clustering.

One example is in the marketing industry. Clustering algorithms have proven to be effective in producing what they call "market segments" in market research. For each market segment, a business may have different criteria for catering to their needs and effectively marketing their product or service.

For example, when plotting customer satisfaction (CSAT) score and customer loyalty (Figure 3.1), clustering can be used to segment the data into subgroups, from which we can get

pretty unexpected results that may stimulate experiments and further analysis. In this case, we attained a whole cluster of customers who are loyal but have low CSAT scores.

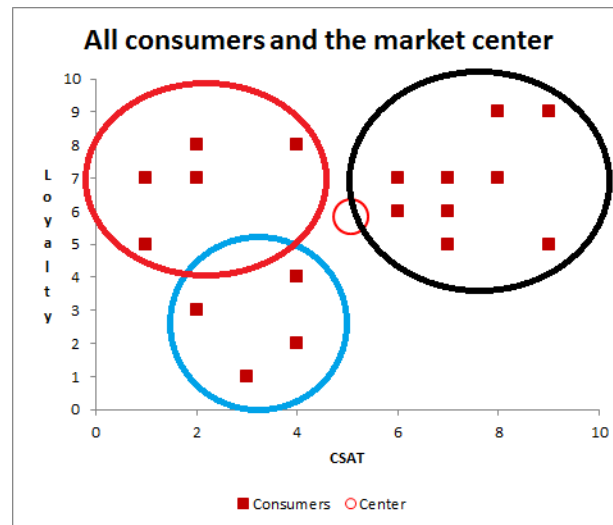


Figure 3.1: Visual from Segmentation Study Guide

Clustering algorithms — particularly k -means ($k=2$) clustering— have also helped speed up spam email classifiers and lower their memory usage. They have also made headway in helping classify different species of plants and animals, organizing of assets, identifying frauds, and studying housing values based on factors such as geographic location.

Let us proceed and discuss a significant method of clustering called hierarchical cluster analysis (HCA). This article will assume some familiarity with k -means clustering, as the two strategies possess some similarities, especially with regard to their iterative approaches.

Hierarchical Clustering

HCA is a strategy that seeks to build a **hierarchy** of clusters that has an established ordering from top to bottom. K -means would not fall under this category as it does not output clusters in a hierarchy, so let's get an idea of what we want to end up with when running one of these algorithms. A "hierarchy of clusters" is usually represented by a **dendrogram**, shown below (Figure 3.2).



Figure 2: Visual from Data Viz Project

For instance, a dendrogram that describes scopes of geographic locations might have a name of a country at the top, then it might point to its regions, which will then point to their states/provinces, then counties or districts, and so on.

Agglomerative vs Divisive Techniques

Hierarchical clustering is a method of clustering that builds a hierarchy of clusters. There are two types of this method.

- **Agglomerative:** This is a bottom-up approach where each observation is treated as its own cluster in the beginning and as we move from bottom to top, each observation is merged into pairs, and pairs are merged into clusters.
- **Divisive:** This is a "top-down" approach: all observations start in one cluster, and splits are performed recursively as we move from top to bottom.

When it comes to analysing data from social networks, hierarchical clustering is by far the most common and popular method of clustering. The nodes (branches) in the graph are compared to each other depending on the degree of similarity that exists between them. By linking together smaller groups of nodes that are related to one another, larger groupings may be created.

The biggest advantage of hierarchical clustering is that it is easy to understand and implement. Usually, the output of this clustering method is analysed in an image such as below. It is called a Dendrogram.

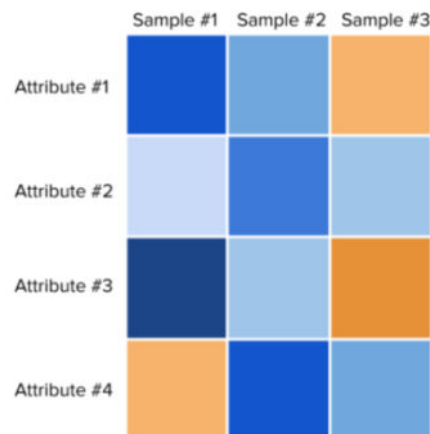
Illustration and Dendrogram

To create a dendrogram, we must compute the similarities between the attributes. Note that to compute the similarity of two features, we will usually be utilizing the Manhattan distance or Euclidean distance. These distances would be recorded in what is called a **proximity matrix**, an example of which is depicted below (Figure 3.3), which holds the distances between each point. We would use those cells to find pairs of points with the smallest distance and start linking them together to create the dendrogram. We will not be delving too much into the mathematical formulas used to compute the distances between the two clusters.

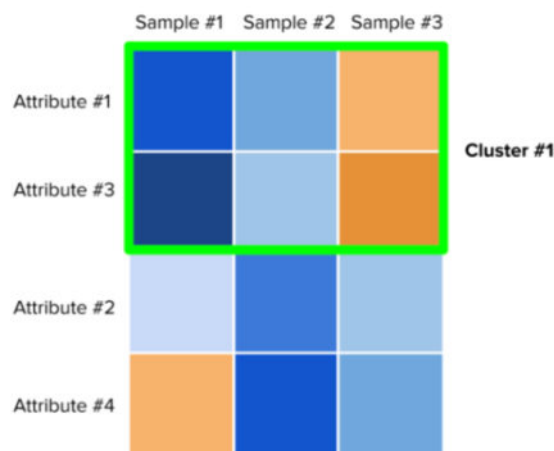
Dist	A	B	C	D	E	F
A	0.00	0.71	5.66	3.61	4.24	3.20
B	0.71	0.00	4.95	2.92	3.54	2.50
C	5.66	4.95	0.00	2.24	1.41	2.50
D	3.61	2.92	2.24	0.00	1.00	0.50
E	4.24	3.54	1.41	1.00	0.00	1.12
F	3.20	2.50	2.50	0.50	1.12	0.00

Figure 3: Visual from Revoledu

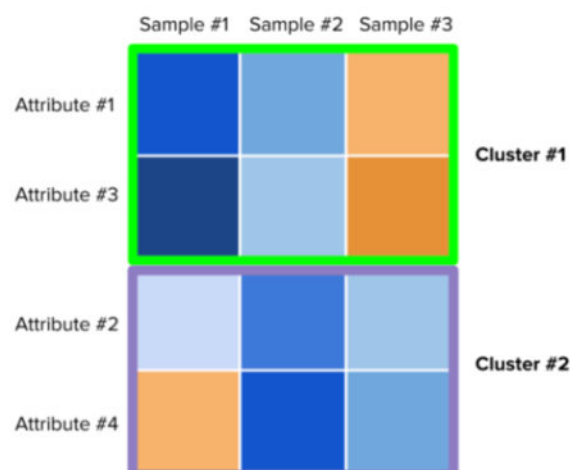
We will describe how a dendrogram is used to represent HCA results in more detail later. For now, consider the following heatmap of our example raw data. We will assume this heat mapped data is numerical. In case you aren't familiar with heatmaps, the different colours correspond to the magnitude of the numerical value of each attribute in each sample. Light colours here, for example, might correspond to middle values, dark orange might represent high values, and dark blue might represent lower values. Darker colours usually refer to extreme values in a numerical dataset.



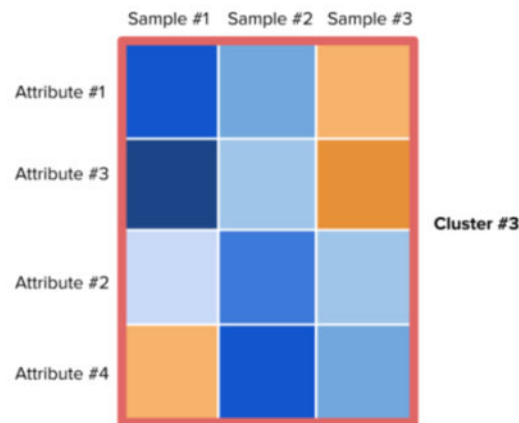
We see that based on the patterns in each row, Attribute #1 and Attribute #3 are similar. We will “cluster” them as follows:



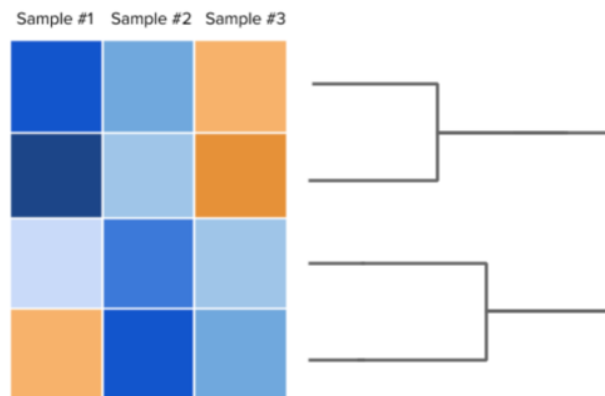
Now, we have a cluster for our first two similar attributes, and we actually want to treat that as one attribute. We now want to figure out which of Attribute #2 and Attribute #4 are most similar to Cluster #1. We then compare the three clusters, but we find that Attribute #2 and Attribute #4 are actually the most similar. Thus, we end up with the following:



Finally, since we now only have two clusters left, we can merge them together to form one final, all-encompassing cluster.



Now, here's how we would summarize our findings in a dendrogram.



Notice the differences in the lengths of the three branches. The original cluster we had at the top, Cluster #1, displayed the most similarity and it was the cluster that was formed first, so it will have the shortest branch. Cluster #2 had the second most similarity and was formed second, so it will have the second shortest branch. The longest branch will belong to the last Cluster #3 since it was formed last.

Hence, the dendrogram indicates both the similarity in the clusters and the sequence in which they were formed, and the lengths of the branches outline the hierarchical and iterative nature of this algorithm. Under the hood, we will be starting with $k=N$ clusters, and iterating through the sequence $N, N-1, N-2, \dots, 1$, as shown visually in the dendrogram.

There are two different approaches used in HCA: **agglomerative** clustering and **divisive** clustering.

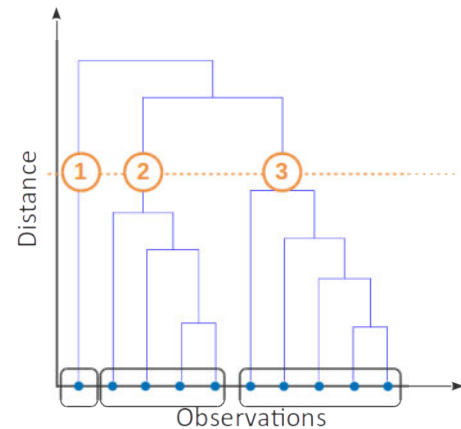
Agglomerative Hierarchical Clustering

This is the more common out of the two approaches, and essentially what I just described above. The process can be summed up in this fashion:

Start by assigning each point to an individual cluster.



At each iteration, we'll merge clusters together and repeat until there is only one cluster left. In this case, the light blue cluster is our last cluster and its branch will be the longest and at the end on the dendrogram.

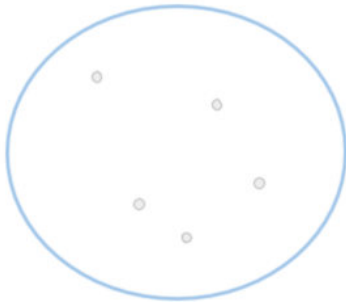


Since each of our observations started in their own clusters and we moved up the hierarchy by merging them together, agglomerative HC is referred to as a **bottom-up** approach.

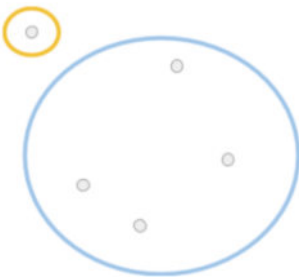
Divisive Hierarchical Clustering

A **top-down** procedure, divisive hierarchical clustering works in reverse order. We start with one cluster, and we recursively split our enveloped features into separate clusters, moving down the hierarchy until each cluster only contains one point. The algorithm is along these lines:

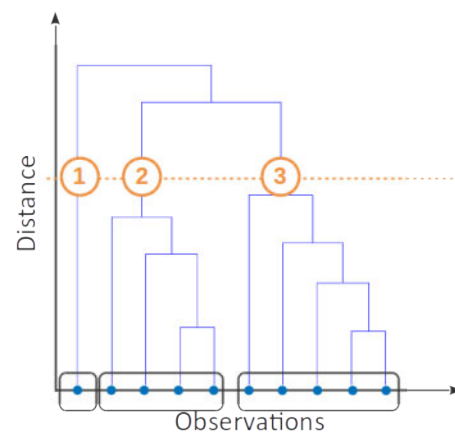
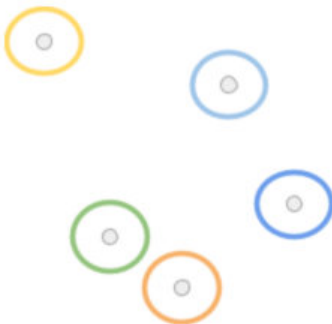
Assign all N of our points to one cluster.

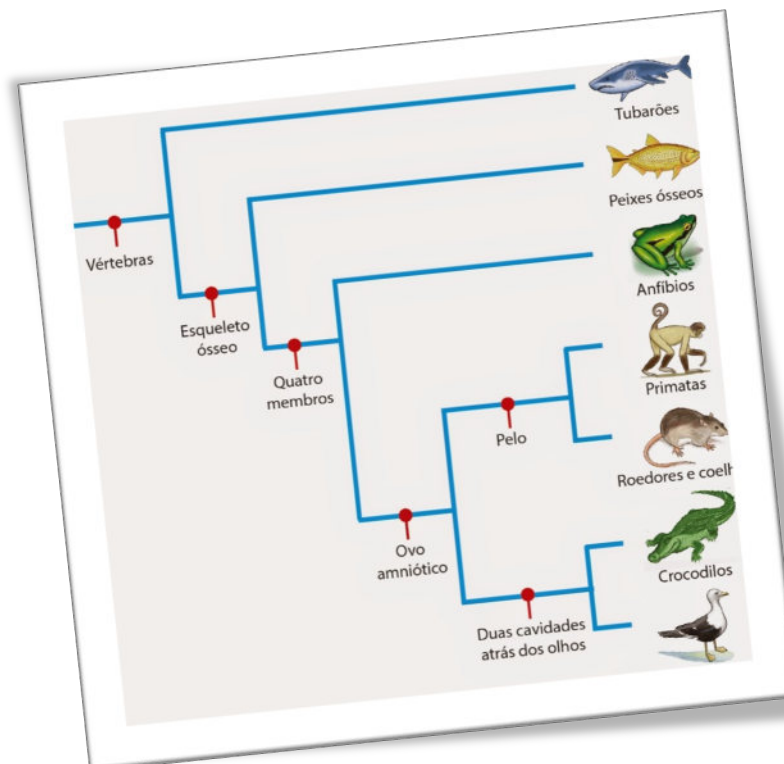


At each iteration, we will split the farthest data point from the rest from this larger cluster and assign it to its own.



This will continue until N singleton clusters remain.





4 — DBSCAN Algorithm

What is DBSCAN

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) is a commonly used unsupervised clustering algorithm proposed in 1996. Unlike the most well known *K-mean*, **DBSCAN** does not need to specify the number of clusters. It can automatically detect the number of clusters based on your input data and parameters. More importantly, DBSCAN can find arbitrary shape clusters that k-means are not able to find. For example, a cluster surrounded by a different cluster.



DBSCAN vs K-means

Also, **DBSCAN** can handle noise and outliers. All the outliers will be identified and marked without been classified into any cluster. Therefore, **DBSCAN** can also be used for Anomaly Detection (**Outlier Detection**)

We need to first understand some basic concepts and terms. *Eps*, *Minpits*, *Directly density-reachable*, *density-reachable*, *density-connected*, *core point* and *border point*

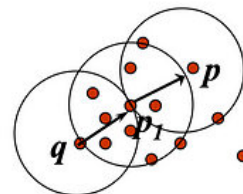
First of all, there are two parameters we need to set for **DBSCAN**, *Eps*, and *MinPts*.

- *Eps*: Maximum radius of the neighbourhood
- *MinPts*: Minimum number of points in an *Eps*-neighbourhood of that point

And there is the concept of **Directly density-reachable**: A point p is directly density reachable from a point q w.r.t. *Eps*, *MinPts*, if $N_{Eps}(q) = \{p \text{ belongs to } D \mid \text{dist}(p,q) \leq Eps\}$ and $|N_{Eps}(q)| \geq \text{MinPts}$. Let's take a look at an example with *Minpts* = 5, *Eps* = 1. Let's take a look at an example to understand density-reachable and density-connected.

■ Density-reachable:

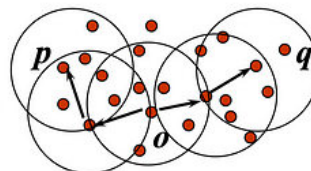
- A point p is **density-reachable** from a point q w.r.t. *Eps*, *MinPts* if there is a chain of points $p_1, \dots, p_n, p_1 = q, p_n = p$ such that p_{i+1} is directly density-reachable from p_i



Density-reachable example

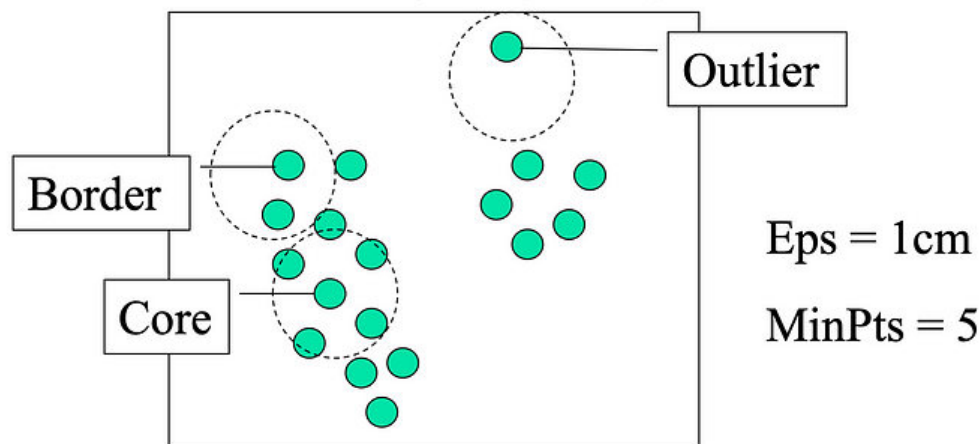
■ Density-connected

- A point p is **density-connected** to a point q w.r.t. *Eps*, *MinPts* if there is a point o such that both, p and q are density-reachable from o w.r.t. *Eps* and *MinPts*



Density-connected example

Finally, a point is a core point if it has more than a specified number of points (*MinPts*) within *Eps*. These are points that are at the interior of a cluster A. And a border point has fewer than *MinPts* within *Eps*, but is in the neighbourhood of a *core point*. We can also define the *outlier* (**noise**) point, which is the points that are neither core nor border points.



Core point, Border point, Outlier Point examples

DBSCAN Steps

Now, let's take a look at how DBSCAN algorithm actually works. Here is the pseudocode.

1. Arbitrary select a point p
2. Retrieve all points density-reachable from p based on Eps and $MinPts$
3. If p is a core point, a cluster is formed
4. If p is a border point, no points are density-reachable from p and DBSCAN visits the next point of the database
5. Continue the process until all of the points have been processed

If a spatial index is used, the computational complexity of **DBSCAN** is $O(n \log n)$, where n is the number of database objects. Otherwise, the complexity is $O(n^2)$.

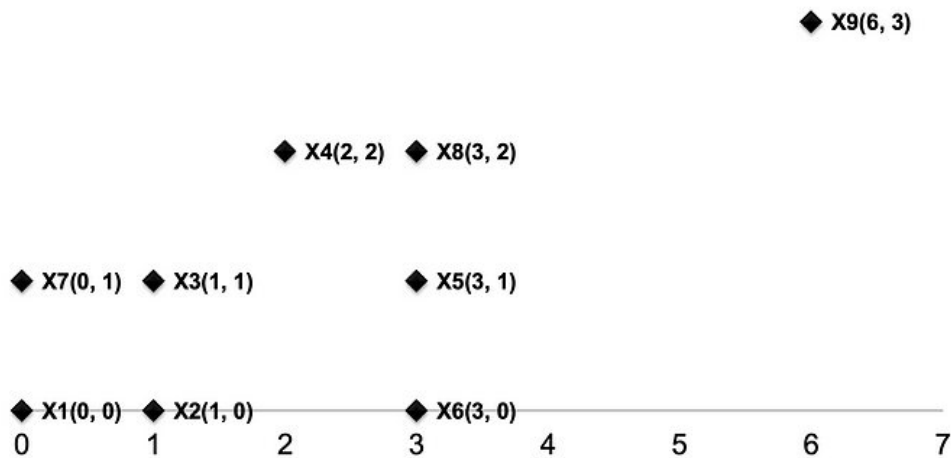
Example

Consider the following 9 two-dimensional data points:

$x_1(0,0)$, $x_2(1,0)$, $x_3(1,1)$, $x_4(2,2)$, $x_5(3,1)$, $x_6(3,0)$, $x_7(0,1)$, $x_8(3,2)$, $x_9(6,3)$

Use the **Euclidean Distance** with $Eps = 1$ and $MinPts = 3$. Find all **core points**, **border points** and **noise points**, and show the **final clusters** using **DBSCAN** algorithm. Let's show the result step by step.

Data Points



Example Data Visualization

First, Calculate the $N(p)$, Eps-neighborhood of point p

$$N(x_1) = \{x_1, x_2, x_7\}$$

$$N(x_2) = \{x_2, x_1, x_3\}$$

$$N(x_3) = \{x_3, x_2, x_7\}$$

$$N(x_4) = \{x_4, x_8\}$$

$$N(x_5) = \{x_5, x_6, x_8\}$$

$$N(x_6) = \{x_6, x_5\}$$

$$N(x_7) = \{x_7, x_1, x_3\}$$

$$N(x_8) = \{x_8, x_4, x_5\}$$

$$N(x_9) = \{x_9\}$$

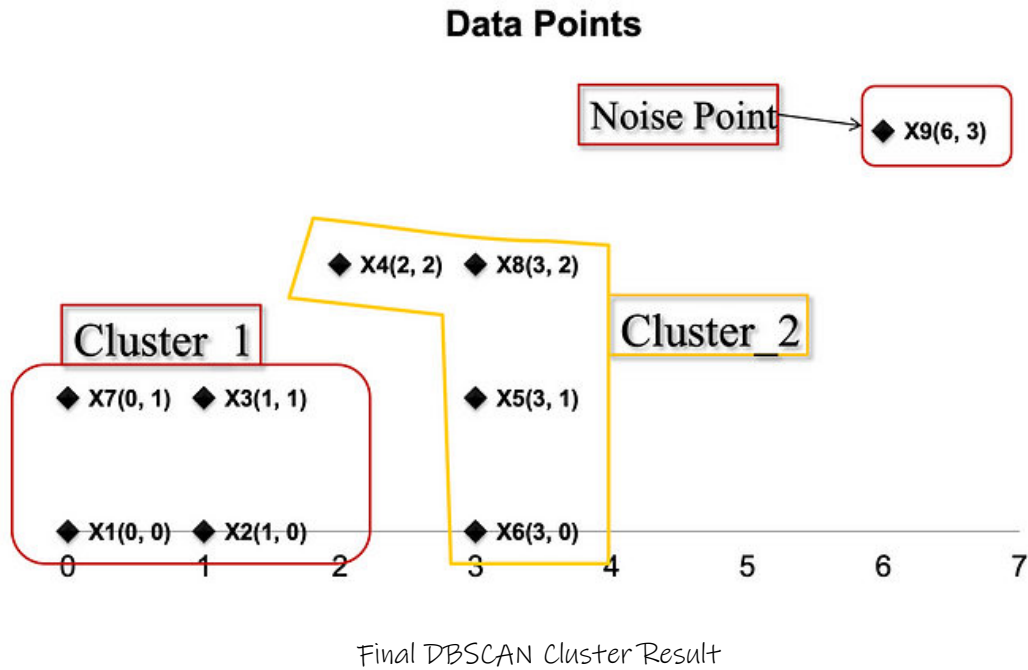
If the size of $N(p)$ is at least MinPts , then p is said to be a core point. Here the given MinPts is 3, thus the size of $N(p)$ is at least 3. **Thus core points are: $\{x_1, x_2, x_3, x_5, x_7, x_8\}$**

Then according to the definition of border points: given a point p , p is said to be a border point if it is not a core point but $N(p)$ contains at least one core point. $N(x_4) = \{x_4, x_8\}$, $N(x_6) = \{x_6, x_5\}$. here x_8 and x_5 are core points, So both **x_4 and x_6 are border points**. Obviously, the point left, **x_9 is a noise point**.

Now, let's follow the preusdecode to produce the clusters.

1. Arbitrary select a point p , now we choose x_1
2. Retrieve all points density-reachable from x_1 : $\{x_2, x_3, x_7\}$
3. Here x_1 is a core point, a cluster is formed. So, we have $\text{Cluster}_1: \{x_1, x_2, x_3, x_7\}$
4. Next, we choose x_5 , Retrieve all points density-reachable from x_5 : $\{x_8, x_4, x_6\}$
5. Here x_5 is a core point, a cluster is formed. So, we have $\text{Cluster}_2: \{x_5, x_4, x_8, x_6\}$

6. Next, we choose x_9 , x_9 is a noise point, noise points do NOT belong to any clusters.
7. Thus, the algorithm stops here.





5 — Other Clustering Algorithms

Mean-Shift

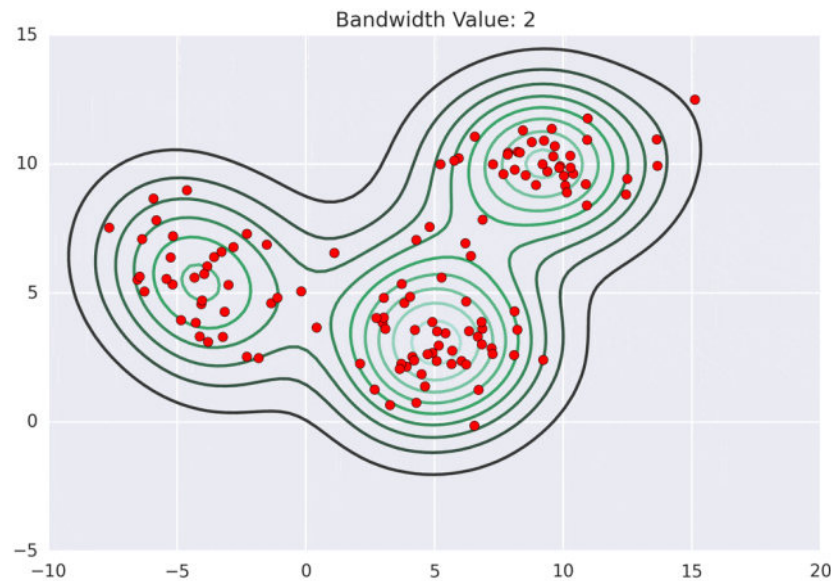
Unlike the K-Means algorithm, *Mean-Shift* algorithm does not require specifying the number of clusters. The algorithm itself automatically determines the number of clusters, which is a pretty big advantage over K-Means if you are not sure about patterns in data.

Mean-Shift is also based on centroids and iteratively assigns each data point to clusters. The most common use case for *Mean-Shift* clustering is image segmentation tasks.

The *Mean-Shift* algorithm is based on kernel density estimation. Similar to the K-Means algorithm, *Mean-Shift* algorithms iteratively assigns each data point towards the closest cluster centroid which are initialized randomly and each point are iteratively moved in the space based on where the most points are i.e. the Mode (mode is the highest density of data points in the region, in the context of the *Mean-Shift*).

This is why the *Mean-Shift* algorithm is also known as the Mode-seeking algorithm. The steps of the *Mean-Shift* algorithm are as follows:

- Pick any random point, and create a window around that random point.
- Calculate the mean of all the points inside this window.
- Shift the window by following the direction of mode.
- Repeat the steps till convergence.



Trajectory of the mean-shift algorithm

1. BIRCH

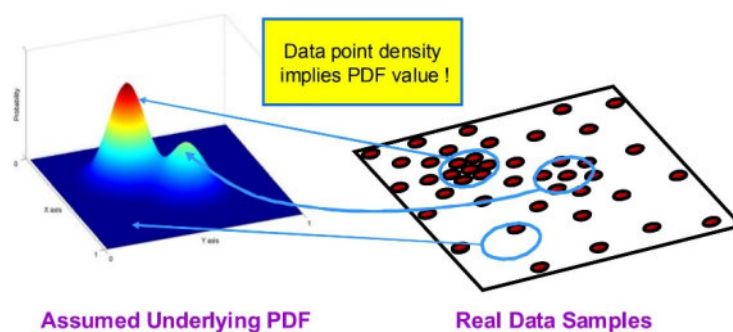
BIRCH stands for Balanced Iterative Hierarchical Based Clustering. It is used on very large datasets where K-Means cannot practically scale. **BIRCH** algorithm divides large data into small clusters and tries to retain the maximum amount of information possible. Smaller groups are then clustered for a final output instead of clustering the large datasets directly.

BIRCH is often used to supplement other clustering algorithms by generating a summary of the information that the other clustering algorithms can utilize. Users have to define the number of clusters for training the **BIRCH** algorithm, similar to how we define it in **K-Means**.

One of the benefits of using **BIRCH** is that it can progressively and dynamically cluster multi-dimensional data points. This is done to create the highest quality clusters under a given memory and time constraints. In most cases, **BIRCH** just needs to do one search across the database, which makes **BIRCH** scalable.

The most common use-case of **BIRCH** clustering algorithm is that it is a memory-efficient alternative to **K-Means** that can be used to cluster large datasets that cannot be handled through **K-Means** due to memory or compute limitations.

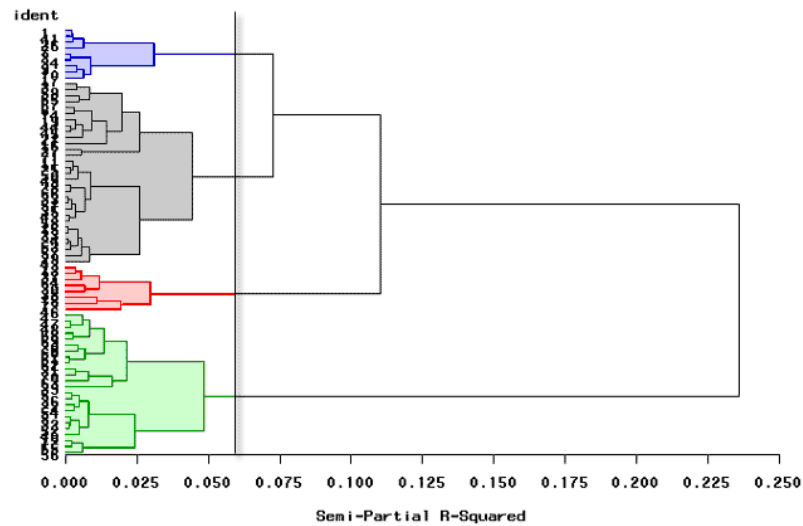
2. Gaussian Mixture Model Algorithm (GMM)



GMM algorithm

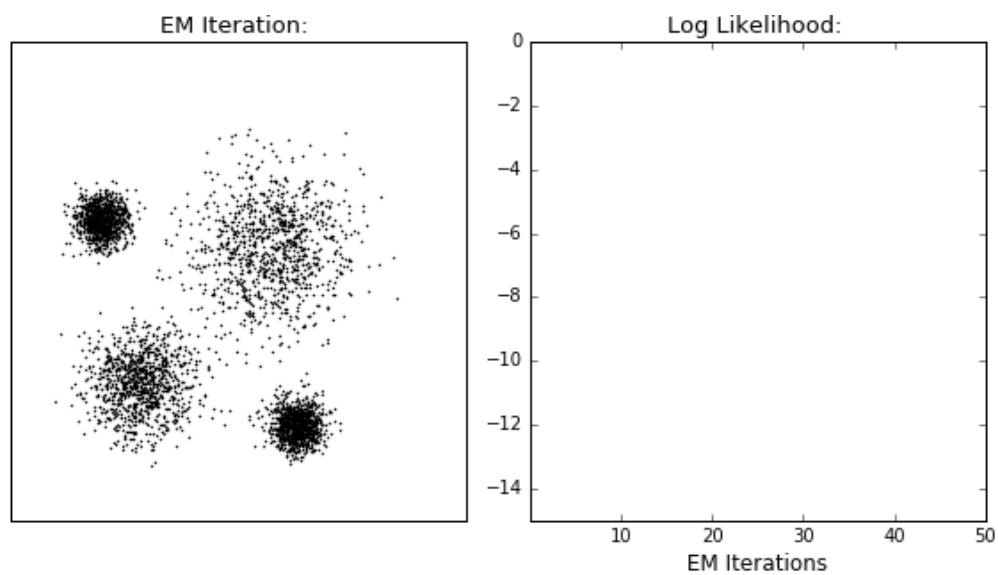
3. Ward Algorithm

Cluster Analysis — Woodyard Hammock — Ward's Method



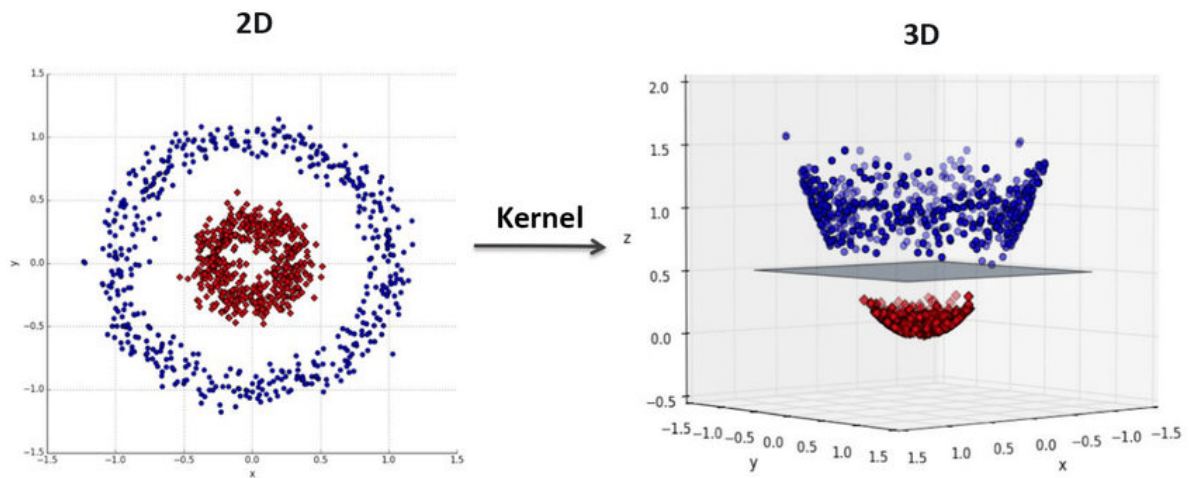
ward algorithm

4. Expectation Maximization (EM) Algorithm



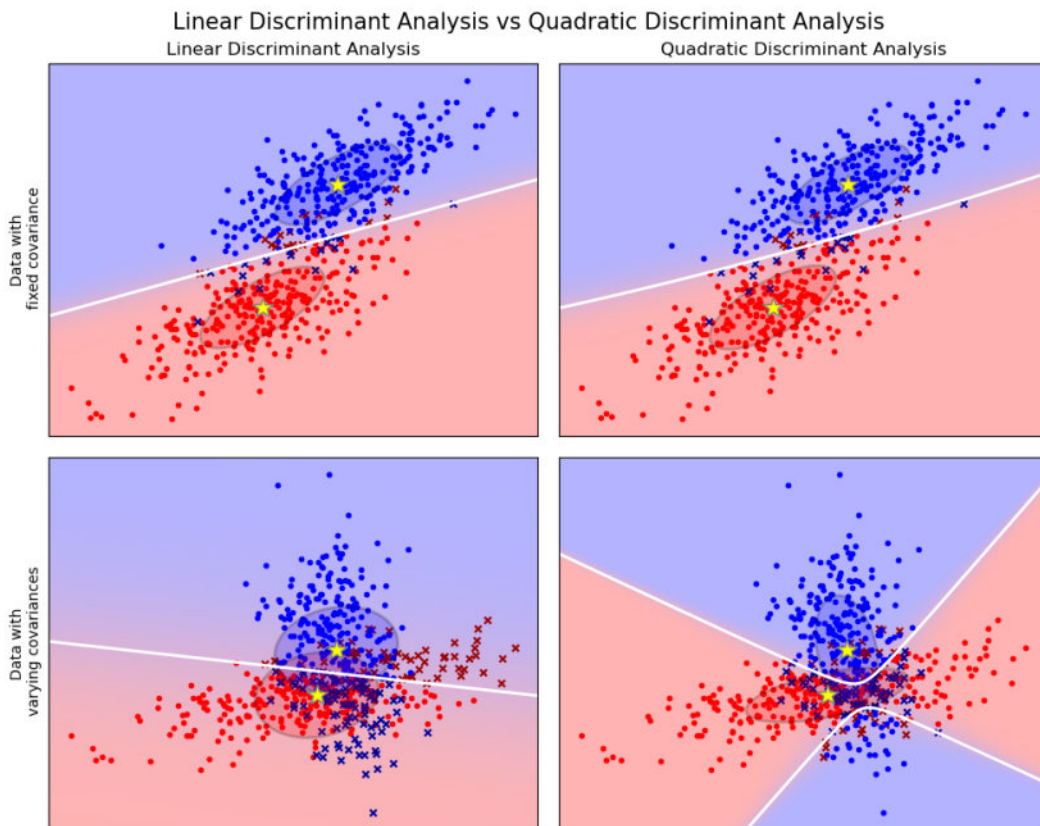
EM algorithm

5. Principal Component Analysis (PCA) Algorithm



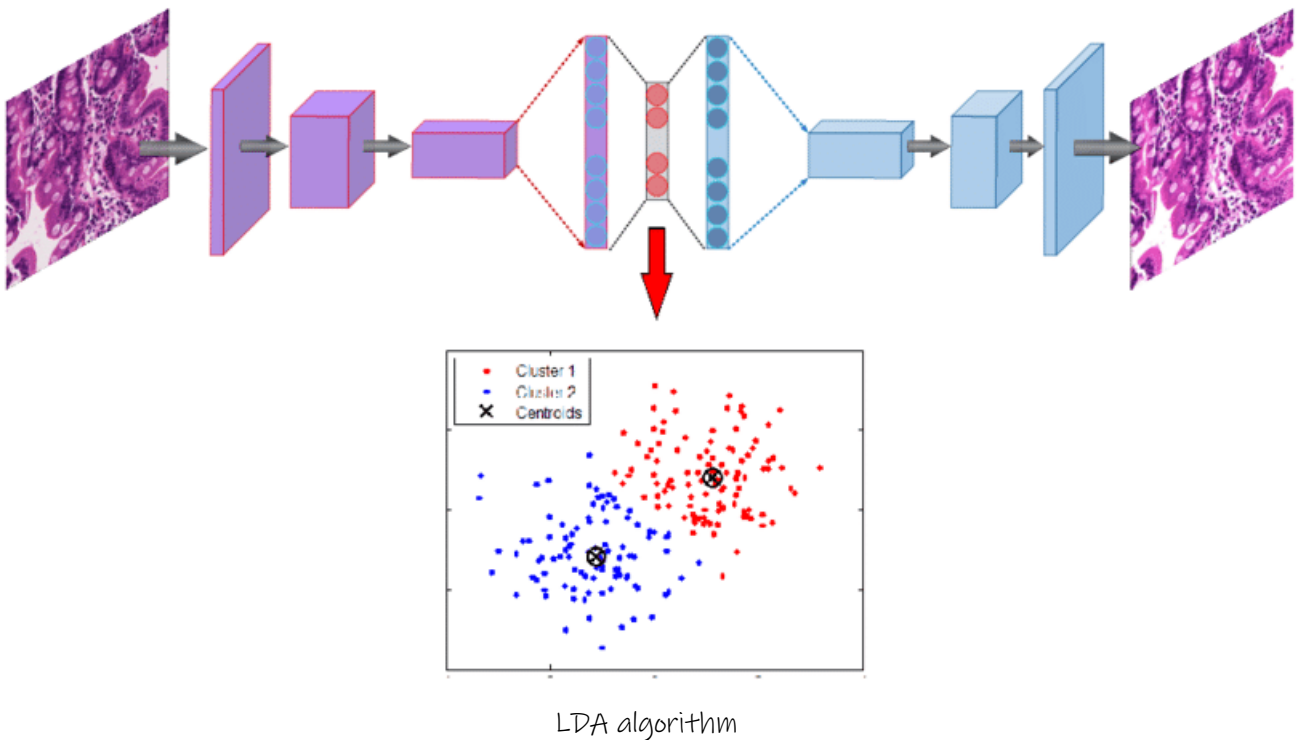
ACP algorithm

6. Linear Discriminant Analysis (LDA) Algorithm



LDA algorithm

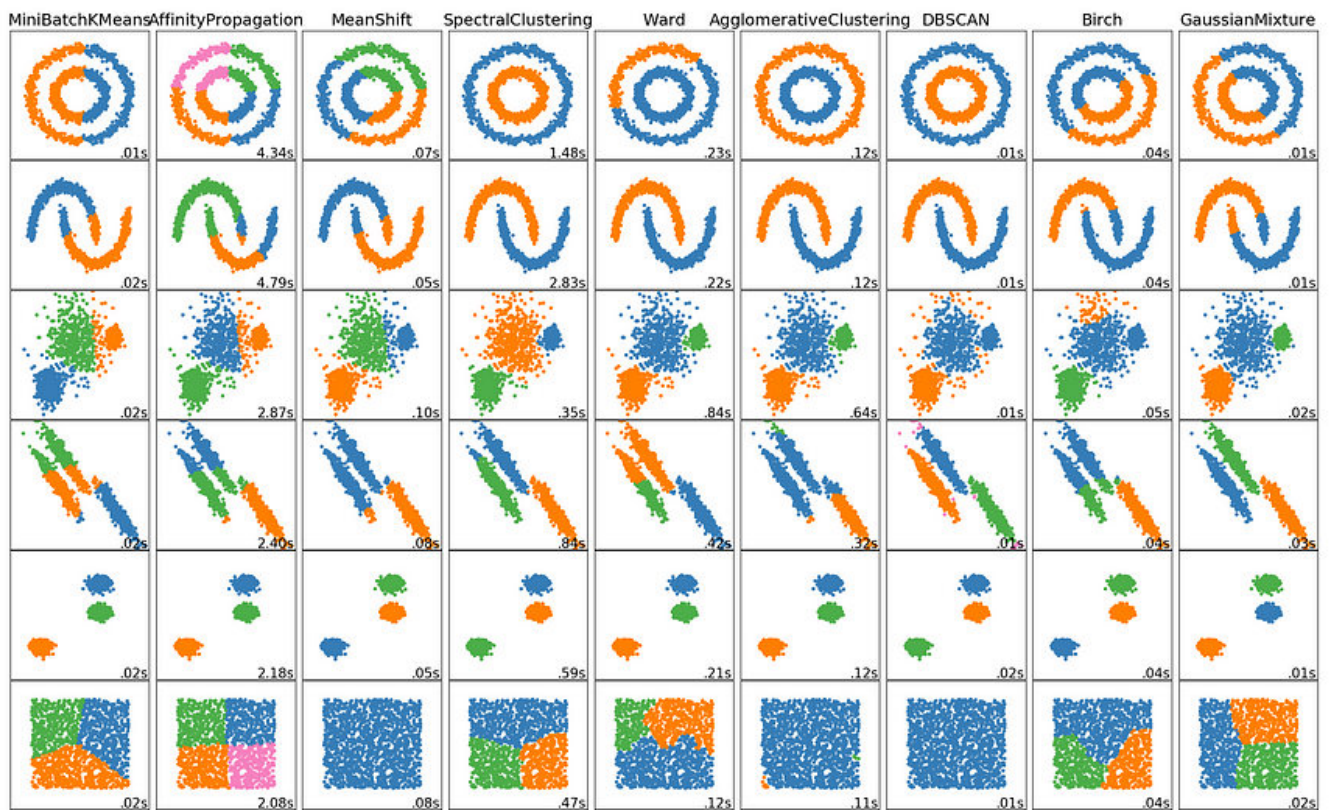
7. Autoencoders



Conclusion

Clustering is a very useful machine learning technique, but it is not as straightforward as some of the supervised learning use-cases like [classification](#) and [regression](#). It is mostly because the performance evaluation and assessing the quality of the model is hard as well as there are some critical parameters such as the number of clusters that the user must define correctly to get meaningful results.

However, there are tons of use-cases of clustering in a wide range of industries, and it is an important skill even for data scientists, machine learning engineers, and data analysts.



Fin